

Secrets Docker en production, sans les passer par SSH

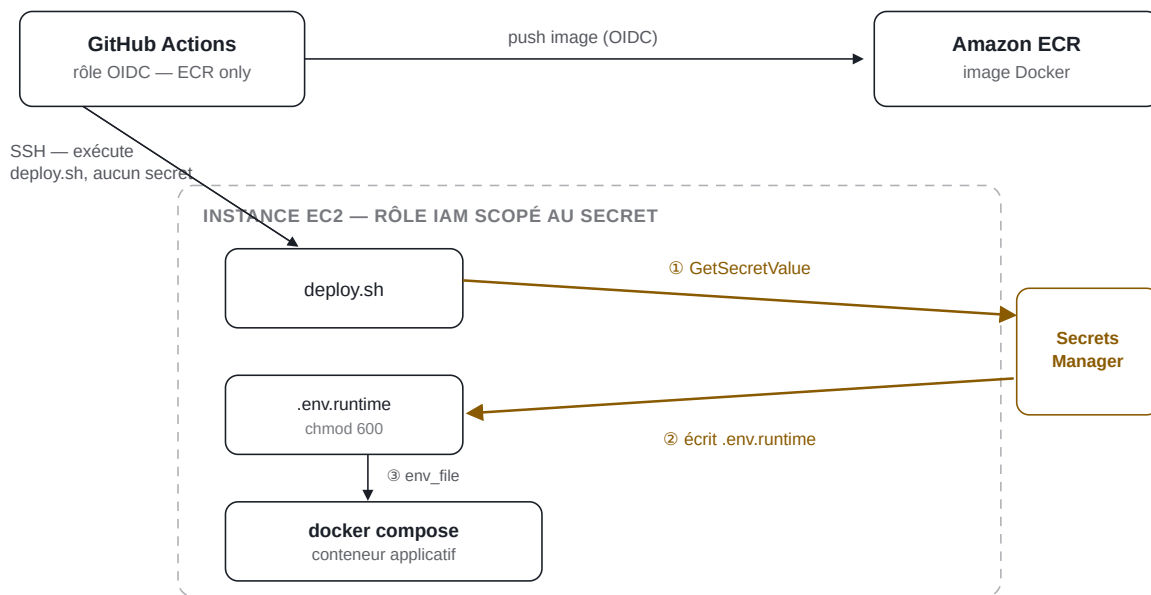
Mode opératoire complet pour charger DB_PASSWORD, PIPELINE_SECRET et les variables MLflow / Airflow dans vos conteneurs Docker déployés depuis GitHub Actions — sans qu'aucune valeur secrète ne transite jamais par le pipeline CI/CD.

POURQUOI ARRÊTER LA COMMANDE SSH ACTUELLE

- Toute variable passée en argument de commande SSH finit dans l'historique bash de la machine cible (`~/.bash_history`) et dans la sortie de `ps aux` pendant l'exécution.
- Dans un workflow GitHub Actions, un secret passé en clair dans une commande apparaît en risque de fuite dans les logs du job si un step l'affiche par erreur (echo, set -x, erreur non gérée).
- Impossible d'auditer qui a lu quelle valeur, ni de faire tourner un secret sans re-déclencher un déploiement manuel.

Le principe

On sépare strictement deux responsabilités. GitHub Actions **déclenche** le déploiement (build de l'image, push, ordre de redémarrage) mais ne voit jamais la valeur des secrets. L'instance EC2 **récupère elle-même** ses secrets dans AWS Secrets Manager, via son propre rôle IAM, au moment du déploiement — exactement comme discuté pour MLflow/Airflow, mais désormais déclenché automatiquement plutôt qu'à la main.



GitHub Actions déclenche `deploy.sh` par SSH sans jamais transporter de secret ; c'est l'instance qui, avec son propre rôle IAM, va chercher la valeur dans Secrets Manager et la sert au conteneur.

- 1 Secret
- 2 Rôle IAM EC2
- 3 `deploy.sh`
- 4 `docker-compose`
- 5 Build image
- 6 OIDC GitHub
- 7 Workflow
- 8 Sans SSH (option)
- 9 Checklist

1 Créer le secret dans AWS Secrets Manager

Un seul secret JSON par environnement, avec toutes les clés dont votre pile a besoin. Pour Airflow, utilisez directement sa convention de nommage `AIRFLOW__SECTION__CLE` : elles arriveront telles quelles dans l'environnement du conteneur.

```
BASH

aws secretsmanager create-secret \
  --name myapp/prod \
  --region eu-west-3 \
  --secret-string '{
    "DB_PASSWORD": "xxxxx",
    "PIPELINE_SECRET": "yyyyy",
    "MLFLOW_TRACKING_URI": "https://mlflow.mondomaine.io",
    "AIRFLOW__CORE__FERNET_KEY": "zzzzz",
```

```
"AIRFLOW__CORE__SQL_ALCHEMY_CONN": "postgresql+psycopg2://airflow:motdepasse@t  
}'
```

Pour mettre à jour une valeur plus tard, `aws secretsmanager put-secret-value` avec le JSON complet — aucune de vos étapes suivantes n'a besoin de changer.

2 Rôle IAM de l'instance EC2

L'instance a besoin de deux droits, chacun scopé au strict nécessaire : lire ce secret précis, et tirer l'image depuis ECR. Le `docker-compose.yml` lui arrive directement par SCP (étape 7), donc pas besoin d'accès S3 côté EC2.

JSON

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "ReadSecret",  
      "Effect": "Allow",  
      "Action": "secretsmanager:GetSecretValue",  
      "Resource": "arn:aws:secretsmanager:eu-west-3:123456789012:secret:myapp/prod",  
    },  
    {  
      "Sid": "EcrAuth",  
      "Effect": "Allow",  
      "Action": "ecr:GetAuthorizationToken",  
      "Resource": "*" ,  
    },  
    {  
      "Sid": "EcrPull",  
      "Effect": "Allow",  
      "Action": [  
        "ecr:BatchGetImage",  
        "ecr:GetDownloadUrlForLayer"  
      ],  
      "Resource": "arn:aws:ecr:eu-west-3:123456789012:repository/myapp"  
    }  
  ]  
}
```

`ecr:GetAuthorizationToken` ne supporte pas de restriction par ressource (exigence AWS, `Resource: "*"` est correct ici) ; l'action suivante, elle, est scopée au repo exact.

Si le secret est chiffré avec une clé KMS gérée par vous (et non la clé par défaut `aws/secretsmanager`), ajoutez aussi `kms:Decrypt` scopée à cette clé précise.

3 Script de déploiement sur l'EC2

Ce script est versionné dans le dépôt Git, au même titre que `docker-compose.yml`, et livré sur l'instance par SCP à chaque déploiement (étape 7) — jamais modifié à la main sur l'EC2. Au moment où il s'exécute, les deux fichiers viennent d'être déposés dans `/opt/myapp`; `deploy.sh` n'a plus qu'à récupérer le secret via le rôle de l'instance (aucune clé AWS à fournir), s'authentifier sur le registre, puis relancer les conteneurs.

```
BASH — /OPT/MYAPP/DEPLOY.SH

#!/usr/bin/env bash
set -euo pipefail

APP_DIR=/opt/myapp
ENV_FILE="$APP_DIR/.env.runtime"
SECRET_ID="myapp/prod"
REGION="eu-west-3"
ECR_REGISTRY="123456789012.dkr.ecr.eu-west-3.amazonaws.com"

# 1. Récupérer les secrets
umask 077
aws secretsmanager get-secret-value \
  --secret-id "$SECRET_ID" \
  --region "$REGION" \
  --query SecretString --output text \
  | jq -r 'to_entries[] | "\(.key)=\(.value)"' > "$ENV_FILE"

chmod 600 "$ENV_FILE"
chown appuser:appuser "$ENV_FILE" "$APP_DIR/docker-compose.yml"

# 2. Authentification au registre puis démarrage
aws ecr get-login-password --region "$REGION" \
  | docker login --username AWS --password-stdin "$ECR_REGISTRY"

cd "$APP_DIR"
docker compose pull
docker compose --env-file "$ENV_FILE" up -d
docker image prune -f
```

Le fichier `.env.runtime` reste sur disque tant que le service tourne (Docker le relit au prochain `up`) mais n'est lisible que par `appuser` et root — ajoutez-le à `.gitignore` par réflexe même s'il n'est pas censé approcher un dépôt. `deploy.sh` ne fait que lire et lancer ce que le workflow vient de déposer : c'est la CI qui décide du tag d'image à déployer (étape 7), jamais le script lui-même.

Provisionner l'instance via le User Data

Le `User Data` EC2 — exécuté une fois, en root, au tout premier boot (cloud-init) — n'a plus qu'à préparer le système : installer Docker, créer l'utilisateur applicatif et le dossier `/opt/myapp`. Il n'écrit plus `deploy.sh` lui-même : ce fichier vit désormais dans Git et arrive par SCP à chaque déploiement (étape 7), comme `docker-compose.yml`.

```
BASH – USER DATA (AMAZON LINUX 2023)

#!/bin/bash
set -euo pipefail

# --- Paquets nécessaires (dnf sur Amazon Linux 2023 ; apt sur Ubuntu) ---
dnf update -y
dnf install -y docker jq
systemctl enable --now docker

mkdir -p /usr/local/lib/docker/cli-plugins
curl -sSL https://github.com/docker/compose/releases/latest/download/docker-compose
  -o /usr/local/lib/docker/cli-plugins/docker-compose
chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

# --- Utilisateur applicatif dédié ---
useradd -r -s /usr/sbin/nologin -d /opt/myapp appuser || true
usermod -aG docker appuser
mkdir -p /opt/myapp
chown appuser:appuser /opt/myapp
```

À coller dans `Advanced details → User data` au lancement (console), ou via `--user-data file://userdata.sh` en CLI / `user_data = file(...)` en Terraform.

TROIS POINTS À NE PAS RATER

- Le rôle IAM (étape 2) doit déjà être attaché à l'instance *au lancement* — le User Data ne peut pas s'attribuer un rôle lui-même. Sans ça, les appels `aws secretsmanager` / `aws ecr` échoueront.
- Le User Data ne s'exécute qu'une fois, au premier boot. Pour le modifier sur une instance déjà lancée, il faut soit relancer une nouvelle instance (le cas normal avec un launch template), soit forcer une ré-exécution (`cloud-init clean --logs && reboot`).

- Sur une instance neuve, ni `deploy.sh` ni `docker-compose.yml` n'existent encore : les deux n'arrivent qu'au premier déploiement déclenché par GitHub Actions (étape 7), qui les dépose par SCP avant de lancer `deploy.sh`.

Durcissement optionnel : lancez l'instance avec `--metadata-options`
`HttpTokens=required,HttpPutResponseHopLimit=1` (IMDSv2 obligatoire, hop limit à 1)
pour qu'un conteneur compromis ne puisse pas interroger le service de métadonnées de l'hôte.

4 `docker-compose.yml`

Ce fichier reste versionné dans le dépôt Git comme gabarit : c'est la CI qui, à chaque déploiement, y injecte le bon tag d'image avant de le livrer sur l'EC2 par SCP (étape 7). Le conteneur consomme le secret via `env_file` — jamais de secret en dur ici ni dans l'image.

```
YAML — /OPT/MYAPP/DOCKER-COMPOSE.YML

services:
  app:
    image: 123456789012.dkr.ecr.eu-west-3.amazonaws.com/myapp:latest
    env_file:
      - .env.runtime
    restart: unless-stopped
    ports:
      - "8000:8000"
```

5 Construire l'image applicative (dossier `app/`)

Le code source est intégré à l'image au moment du build, jamais déployé « à plat » sur le serveur — c'est ce qui donne un artefact immuable, identique en test et en prod. Un `Dockerfile` multi-stage sépare l'installation des dépendances de l'image finale livrée.

C'EST VOTRE DOCKERFILE RÉEL QUI EST CONSTRUIT

La CI ne fournit pas son propre `Dockerfile` : l'étape `docker/build-push-action` du workflow (étape 7) construit exactement le fichier versionné dans votre dépôt, à la racine du contexte checkouté. Le multi-stage ci-dessous est une amélioration

suggérée, pas quelque chose que la CI impose — un Dockerfile existant à un seul stage se construit tel quel, sans rien changer.

Deux points à vérifier de votre côté : que les chemins `COPY` correspondent à la structure réelle de votre dépôt (ce guide suppose un dossier `app/`), et que le fichier soit bien à la racine — sinon, ajoutez `file: ./chemin/vers/Dockerfile` à l'étape `docker/build-push-action` (étape 7).

DOCKERFILE

```
FROM python:3.12-slim AS builder
WORKDIR /build
COPY app/requirements.txt .
RUN pip install --no-cache-dir --target=/deps -r requirements.txt
COPY app/ .

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /deps /usr/local/lib/python3.12/site-packages
COPY --from=builder /build .
USER 1000
CMD ["python", "main.py"]
```

Les outils de build (compilateurs, cache pip) restent dans l'étape `builder` et ne se retrouvent pas dans l'image livrée — plus petite, moins de surface d'attaque.

`.dockerignore`

TEXT — `.DOCKERIGNORE`

```
.git
.env
secret.sh
**/__pycache__
**/*.pyc
tests/
.github/
```

RÈGLE ABSOLUE

- Aucun secret n'entre jamais dans le contexte de build — ni `COPY secret.sh`, ni `ARG` / `ENV` contenant une valeur sensible, même dans une étape intermédiaire du multi-stage : le contenu d'une layer reste extractible même si un stage suivant ne la garde pas. `DB_PASSWORD`, `PIPELINE_SECRET`, etc. n'arrivent que plus tard, au runtime, via `.env.runtime` (étapes 1 à 4).

Docker Hub ou Amazon ECR ?

Le guide pousse par défaut vers ECR via OIDC (étape 6), ce qui évite toute clé longue durée côté GitHub. Docker Hub reste un choix valide — image publique nécessaire, équipe déjà dessus — mais comme il n'existe pas de fédération OIDC pour Docker Hub, il faut alors stocker un identifiant en secret GitHub : un **access token scopé** (Docker Hub → Account Settings → Security → New Access Token, jamais le mot de passe du compte), limité en droits au repo concerné. L'étape 7 montre les deux options.

À considérer en plus : un scan de vulnérabilités (Trivy, `docker scout`) sur l'image juste avant le push, pour faire échouer le build si une CVE critique est détectée dans les dépendances de `app/`.

SAUVEGARDE DU CODE SOURCE SUR S3

Conserver une copie de `app/` sur S3 à chaque build, en plus de Git, est légitime pour de l'audit ou une sauvegarde hors dépôt — mais c'est un chemin parallèle au déploiement, pas une étape dont l'EC2 dépend : le `docker-compose.yml` voyage par SCP (étape 7), l'EC2 n'a même pas accès à ce bucket. Voir l'étape 7 pour ajouter la sauvegarde au workflow.

6 GitHub Actions → AWS sans clé statique (OIDC)

Plutôt que de stocker `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` dans les secrets GitHub, on fédère l'identité du workflow directement à un rôle IAM éphémère. Aucune clé longue durée à faire fuiter ou à faire tourner.

Créer le fournisseur OIDC (une seule fois par compte)

BASH

```
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com \
  --thumbprint-list 6938fd4d98bab03faadb97b34396831e3780aea1
```

Rôle assumé par le workflow, restreint au dépôt et à la branche

JSON — TRUST POLICY

```
{
  "Version": "2012-10-17",
```

```

"Statement": [
  {
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.github
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {
        "token.actions.githubusercontent.com:sub": "repo:mon-org/mon-repo:ref:re
      }
    }
  }
]
}

```

OÙ AJOUTER CETTE POLICY

Le JSON ci-dessus est la **trust policy** du rôle (« qui a le droit de l'endosser ») — à ne pas confondre avec la **permissions policy** juste en dessous (« ce que le rôle peut faire une fois endossé »). Les deux vivent sur le même rôle IAM, mais dans deux onglets différents.

Console : IAM → Roles → *Create role* → type d'entité de confiance *Web identity* → sélectionnez le fournisseur `token.actions.githubusercontent.com` créé juste au-dessus → audience `sts.amazonaws.com` . Une fois le rôle créé, onglet *Trust relationships* → *Edit trust policy* → collez le JSON exact (avec la condition `StringLike`) → *Update policy*.

CLI, en une fois à la création :

BASH

```

aws iam create-role \
  --role-name github-actions-deploy \
  --assume-role-policy-document file://trust-policy.json

```

La policy de permissions, elle, s'attache séparément sur ce même rôle (onglet *Permissions* en console, ou `put-role-policy` en CLI) et se limite à `ecr:GetAuthorizationToken` , `ecr:BatchCheckLayerAvailability` , `ecr:PutImage` , `ecr:InitiateLayerUpload` , `ecr:UploadLayerPart` , `ecr:CompleteLayerUpload` sur le repo ECR concerné. Si vous activez la sauvegarde optionnelle de `app/` (étape 7), ajoutez `s3:PutObject` scoping à `arn:aws:s3:::myapp-deploy/myapp/app/*` — en écriture seule, ce

rôle n'a jamais besoin de relire ce qu'il y dépose. Ce rôle n'a **pas** accès à Secrets Manager — seule l'instance EC2 l'a (étape 2), et le `docker-compose.yml` ne transite par aucun des deux : il voyage directement par SCP (étape 7).

BASH

```
aws iam put-role-policy \  
  --role-name github-actions-deploy \  
  --policy-name github-actions-ecr \  
  --policy-document file://permissions-policy.json
```

Le nom du rôle (`github-actions-deploy`) doit correspondre exactement à celui référencé dans `role-to-assume` du workflow (étape 7).

7 Le workflow GitHub Actions complet

Build et push de l'image via le rôle OIDC, sauvegarde optionnelle du code source sur S3, rendu du `docker-compose.yml` avec le bon tag, copie directe de ce fichier et de `deploy.sh` sur l'EC2 par SCP, puis déclenchement du déploiement par SSH — ni la connexion SCP ni la connexion SSH ne transportent de secret applicatif.

`deploy.sh` est ainsi écrasé à chaque déploiement par la version présente dans Git à ce commit — c'est voulu : la seule source de vérité est le dépôt, aucune modification à la main sur l'EC2 ne doit survivre au déploiement suivant. Comme il s'exécute en root via `sudo`, toute modification de ce fichier passe par la revue de code habituelle (pull request) avant de fusionner sur la branche qui déclenche le déploiement.

YAML — `.GITHUB/WORKFLOWS/DEPLOY.YML`

```
name: Deploy  
  
on:  
  push:  
    branches: [main]  
  
permissions:  
  id-token: write  
  contents: read  
  
jobs:  
  build-and-deploy:  
    runs-on: ubuntu-latest  
    environment: production  
    steps:  
      - uses: actions/checkout@v4
```

```

- name: Auth AWS via OIDC
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123456789012:role/github-actions-deploy
    aws-region: eu-west-3

- name: Login ECR
  uses: aws-actions/amazon-ecr-login@v2

- name: Build et push de l'image
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: 123456789012.dkr.ecr.eu-west-3.amazonaws.com/myapp:${{ github.sha }}
    cache-from: type=gha
    cache-to: type=gha,mode=max

- name: Sauvegarde du code source (optionnel, hors Git)
  run: |
    tar czf app.tar.gz app/
    aws s3 cp app.tar.gz s3://myapp-deploy/myapp/app/${{ github.sha }}.tar.gz

- name: Rendre le docker-compose.yml avec le tag de cette build
  run: |
    sed -i "s|image:.*|image: 123456789012.dkr.ecr.eu-west-3.amazonaws.com/myapp:${{ github.sha }}" docker-compose.yml

- name: Copier docker-compose.yml et deploy.sh sur l'EC2 (SCP, aucun secret)
  uses: appleboy/scp-action@v0.1.7
  with:
    host: ${ secrets.EC2_HOST }
    username: ${ secrets.EC2_USER }
    key: ${ secrets.EC2_SSH_KEY }
    source: "docker-compose.yml,deploy.sh"
    target: "~"

- name: Déploiement (SSH, aucun secret transmis)
  uses: appleboy/ssh-action@v1
  with:
    host: ${ secrets.EC2_HOST }
    username: ${ secrets.EC2_USER }
    key: ${ secrets.EC2_SSH_KEY }
    script: |
      sudo mv ~/docker-compose.yml /opt/myapp/docker-compose.yml
      sudo mv ~/deploy.sh /opt/myapp/deploy.sh
      sudo chmod +x /opt/myapp/deploy.sh

```

```
sudo chown appuser:appuser /opt/myapp/deploy.sh
sudo /opt/myapp/deploy.sh
```

CE QUE GITHUB ACTIONS MANIPULE ICI

- `EC2_HOST` , `EC2_USER` , `EC2_SSH_KEY` : des identifiants *d'accès au serveur*, stockés en secrets GitHub chiffrés — pas des secrets applicatifs. Ils ne donnent aucun accès direct à `DB_PASSWORD` ou `PIPELINE_SECRET` .
- Le tag d'image (`github.sha`), jamais `:latest` en production — ça vous garantit de pouvoir revenir en arrière précisément.

Variante Docker Hub

Si vous poussez vers Docker Hub plutôt qu'ECR (voir l'étape 5), remplacez les steps

`Auth AWS via OIDC` et `Login ECR` par :

YAML

```
- name: Login Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${ secrets.DOCKERHUB_USERNAME }
    password: ${ secrets.DOCKERHUB_TOKEN }
```

Puis changez le tag de l'image dans le step de build en `monorg/myapp:${ github.sha }`. `DOCKERHUB_TOKEN` est un access token scopé, pas le mot de passe du compte.

8 Version renforcée : supprimer complètement SSH

Si vous voulez aller plus loin, remplacez la clé SSH et le port 22 ouvert par `aws ssm send-command` , exécuté par le même rôle OIDC. Plus de clé privée à stocker, plus de port entrant à exposer — la commande passe par l'API AWS, authentifiée et journalisée dans CloudTrail.

YAML — REMPLACE LE STEP SSH

```
- name: Déploiement via SSM
  run: |
    aws ssm send-command \
      --instance-ids "i-0123456789abcdef0" \
      --document-name "AWS-RunShellScript" \
```

```
--parameters commands="/opt/myapp/deploy.sh" \  
--region eu-west-3
```

Prérequis : l'agent SSM tourne sur l'instance (préinstallé sur les AMI Amazon Linux récentes), le rôle de l'instance a la policy managée

`AmazonSSMManagedInstanceCore`, et le rôle GitHub Actions gagne l'action

`ssm:SendCommand` scoping à cette instance.

9 Checklist sécurité

- ✓ **Rôle IAM de l'instance** scopé à l'ARN exact du secret, pas à `secret:*`.
- ✓ **Rôle IAM du workflow GitHub** restreint par `sub` (repo + branche exacts) et sans accès à Secrets Manager.
- ✓ **Rotation** activée sur le secret (Secrets Manager peut la faire automatiquement pour RDS/PostgreSQL/MySQL).
- ✓ **Environment protection rule** GitHub sur `production` (approbation manuelle avant déploiement si souhaité).
- ✓ **Aucun** `echo` ou `set -x` dans `deploy.sh` ni dans le workflow, pour ne jamais faire fuiter une valeur dans les logs.
- ✓ **Alerte CloudTrail** sur les appels `secretsmanager:GetSecretValue` hors de l'instance attendue.
- ✓ **Volume EBS chiffré** et snapshots non partagés publiquement.
- ✓ **Clé SSH de déploiement** dédiée (pas votre clé personnelle), restreinte par groupe de sécurité, ou remplacée par SSM (étape 7).
- ✓ **Tag d'image immuable** (`github.sha`), jamais `:latest` en production.
- ✓ **.dockerignore à jour** — aucun secret, `.git` ou fichier de test dans le contexte de build.
- ✓ **Image scannée** (Trivy, `docker scout`) avant push — build cassé sur CVE critique.

Régions et identifiants (compte `123456789012`, région `eu-west-3`, dépôt `mon-org/mon-repo`) sont des exemples à remplacer par les vôtres.